

Borla

Technical Debt Plan

Student: **George Boadu Junior**

Student ID: **22427354**

Course: CSCD 602 — Advanced Software Engineering, University of Ghana

Document: Technical_Debt_Plan.pdf · Version 1.0 · 13 August 2026

1. How debt was tracked during the build

Every deliberate shortcut was written down **as the decision was made**, not reconstructed afterwards — most of the items below are quoted almost verbatim from code comments left at the exact line where the trade-off was taken (e.g. `server/src/jobs/index.ts`, `server/src/modules/auth/routes.ts`). This is the practice the course asks for: technical debt as a first-class, visible decision, not a silent shortcut discovered later.

Each item below follows **Debt** → **Cause** → **Impact** → **Priority** → **Proposed Resolution**, and is labelled as one of:

- **Acceptable temporarily** — correct trade-off for a graded demo at pilot scale; revisit only if the project continues past this exam.
- **Scheduled for future resolution** — fine for now, but must be fixed before real users or a meaningful scale-up.
- **Critical** — must not reach real users in its current form.

2. Debt register

ID	Debt	Cause	Impact	Priority	Proposed resolution
TD-01	No AI key by default — resolved: a real <code>GEMINI_API_KEY</code> is now set (Render + local <code>.env</code> for testing), and moderation actually calls Gemini instead of falling straight to the manual queue. A related defect (D-08) surfaced the moment the key went live: the hardcoded model ID (<code>gemini-2.5-flash</code>) returned <code>404</code> — no longer available to new users for this key's account tier, so every call silently fell back to fail-closed/manual. First fix attempt (<code>gemini-flash-latest</code> alone) still wasn't confirmed live; root-caused by testing all six plausible model IDs directly against the real key — <code>gemini-2.5-flash</code> / <code>gemini-2.0-flash</code> both <code>404</code> , <code>gemini-3.6-flash</code> / <code>gemini-3.5-flash</code> / <code>gemini-3.7-flash</code> / <code>gemini-flash-latest</code> all work. Rewritten to try an ordered candidate list and cache whichever one succeeds per process, so the <i>next</i> Gemini deprecation degrades to "skip to the next candidate" instead of silently going dark again.	No Google AI Studio account was provisioned for the <i>original</i> build; the moderation pipeline (<code>server/src/ai/moderation.ts</code>) checks <code>GEMINI_API_KEY</code> and no-ops if absent — that fail-closed behaviour is exactly why D-08 degraded silently to the manual queue instead of breaking the review flow.	Confirmed end-to-end against the real key, locally: <code>classifyText("Great pickup, right on time, very professional!", "review")</code> returned <code>{"verdict":"allow","confidence":0.99,...}</code> , a real classification, not a fallback null.	● Resolved	Re-run the same check once directly against production (a review, confirm it publishes without landing in the manual queue) — the code path is <i>proven</i> this is now just confirming the <i>deployed</i> process has the same <i>var</i> .
TD-02	No real SMS gateway — mostly resolved: <code>server/src/utis/sms.ts</code> sends the OTP via GiantSMS when <code>GIANTSMS_API_TOKEN</code> / <code>GIANTSMS_SENDER_ID</code> are configured, falling back to the in-app <code>devOtp</code> display only if the send fails or isn't configured. Confirmed live in production: <code>POST /api/auth/otp/request</code> against <code>https://borla.onrender.com</code> returned <code>{"delivered":true}</code> — GiantSMS accepted the reconstructed request shape (auth header, <code>from</code> / <code>to</code> / <code>msg</code> body) for real, not just in theory. A related gap closed at the same time: the two seeded demo numbers are arbitrary, not real handsets, so once real SMS was wired up they would have started reporting <code>delivered:true</code> while the code vanished into a gateway with no phone behind it — a graded demo would have been locked out. Fixed by special-casing <code>DEMO_PHONES</code> (<code>server/src/seed.ts</code>) to always use a fixed, documented code (<code>DEMO_OTP</code>) and never attempt a real send, regardless of gateway state.	The HTTP contract was reconstructed from published third-party client libraries, not a first-party OpenAPI spec. Production accepted the request; actual handset delivery to a <i>real</i> number has still not been visually confirmed (by definition, the two numbers this build can freely test against are the exempted demo ones).	Low residual risk: the gateway accepting the request end-to-end (not just returning a generic <code>200</code>) is strong evidence the integration is correct for a genuine phone number, but "accepted by the gateway" and "arrived on a phone" are not identical.	● Scheduled (down from ● Critical — a real send now provably works at the HTTP layer, and grading no longer depends on it working at all)	Confirm one delivery to a real handset, then remove the <code>delivered</code> fallback field for non-demo numbers entirely so a failed surface as a retryable error of a security bypass.
TD-03	Native background geolocation was not built. Collectors only report position while their browser tab is open and in the foreground (<code>useGeolocation</code> <code>watchPosition</code> , no Capacitor/native service).	Requires an Android toolchain, a background-location plugin (commercial or complex to configure), and a physical/emulated device to validate "screen-off roaming" — none available, and a native APK isn't gradable as a "Live Application URL" per the exam's own submission format.	This is the single biggest gap from the original design's own risk register (<code>borla-technical-design.md</code> §14, risk #1) — a collector who backgrounds the app or locks their phone stops updating position, which is exactly the failure mode that erodes trust in the map.	● Critical (for any real pilot) / ● Acceptable for this exam	Wrap the existing shared React Native screens in Capacitor, add <code>@capacitor-community/background-geolocation</code> , and test on real Android hardware (Tecno/Infinix/itel) per the risk mitigation — no server-change needed, since the API already accepts position updates from any authenticated client.
TD-04	No offline PWA shell — mostly resolved: <code>vite-plugin-pwa</code> now ships a real installable PWA — manifest + icons (<code>client/public/icon-*.png</code>), a Workbox service worker precaching the app shell (12 entries, ~490KB), an explicit "Update available" banner (<code>registerType: 'prompt'</code> , <code>src/pwa/UpdatePrompt.tsx</code> — never silently swaps content under an open tab) and a real "Install app" button	The remaining gap is genuinely more work (background sync / an offline action queue), not a missing dependency.	Households still see a blank state for anything requiring a live fetch while offline (map data, requests, broadcasts) — the shell now loads, the <i>data</i> doesn't.	● Acceptable (down from ● Scheduled — the install/update half is done; only the	Add a Workbox Background queue (or a hand-rolled IndexedDB) for "clear pin" and similarly small mutating actions that survive a dropped connection.

ID	Debt	Cause	Impact	Priority	Proposed resolution
TD-11	Photo → waste-type AI classification was not built (§18 Job B in the original design).	Explicitly deprioritised behind moderation (Job A) per the original design's own value ranking — moderation is the safety-critical one; photo classification is a UX nicety.	Households still pick a waste-type tile manually instead of snapping a photo — a real but non-blocking gap for low-literacy accessibility.	🟡 Scheduled	Reuse the existing 'aiClient' shaped moderation module' pattern: a second thin wrapper calling Gemini's multimodal endpoint, wired to a new <code>POST /broadcasts</code> image field, client side downscaled before upload.
TD-12	Provider call-masking was not built — accepted requests reveal each party's real phone number ("reveal-on-accept", the original design's own recommended v1 approach, §10 option 1).	This is the design's own recommended v1 trade-off, not a shortcut taken under exam pressure — kept as-is deliberately.	Real phone numbers are exchanged between two people who chose each other via an accepted request — acceptable given the low stakes described in the original design.	🟢 Acceptable	Add provider number-mask (e.g. via Hubtel) only if abuse reports or scale warrant the per-call cost, exactly as the original design already prescribes.
TD-13	Single Render Web Service, no staging environment, no CI pipeline beyond local <code>npm test</code> .	Free-tier, single-service deploy was the fastest path to a real "Live Application URL" within the exam window.	A bad commit reaches production directly; no automated gate before deploy.	🟡 Scheduled	Add a GitHub Actions workflow running <code>npm test</code> on every commit (config-only change, no app change needed) and a second Render environment for staging before promoting to the production URL.
TD-14	The collector↔-household route (once a direct request is accepted) calls OSRM's public demo routing server (<code>router.project-osrm.org</code>) directly from the browser — no API key, but also no SLA, rate-limit guarantee, or uptime commitment.	Zero-config, matching the same trade-off already made for the OSM raster tile layer (TD-08): a routing feature needed to exist for this build, and every account-gated alternative (Mapbox Directions, Google Directions) needed a key nobody had time to provision.	If the demo server is down, slow, or rate-limits this app's traffic, the route silently degrades to a straight line between the two points (<code>client/src/components/MapView.tsx</code> , <code>useRoute</code> 's fallback) rather than failing — correct fail-safe behaviour, but the <i>quality</i> of the route (does it actually follow real roads) is not guaranteed at any given moment.	🟢 Acceptable	If usage grows, move to a self-hosted OSRM instance or a commercial Directions API with an SLA. <code>useRoute</code> hook's interface (<code>from</code> / <code>to</code> in, a lat/lon pair, distance/duration out) would need to change, only the fetch inside it.

3. Debt NOT taken — deliberate design choices worth naming

To be explicit about the difference between *debt* (a shortcut that should eventually be paid down) and a *considered trade-off that stays*: the no-collector-binding broadcast plane, the absence of in-app payments, and reveal-on-accept-instead-of-provider-masking are not technical debt — they are the original design's own architecture, kept unchanged because they are still the right call at this scale (see `borla-technical-design.md` §1 and §10, and TD-12 above).

4. Repayment plan by phase

This maps directly onto `Project_Documentation.md` §15 (Maintenance & Future Evolution):

Phase	Items paid down
v1.1 (immediate post-submission, if continued)	TD-06 (admin 2FA) — the one remaining 🟢 Critical item, must land before any real admin touches the system; confirm one real GiantSMS handset delivery to fully close TD-02
v1.2	TD-13 (CI pipeline)
v1.3	TD-03 (native Capacitor collector app + background geolocation) — the single biggest lift, matched to the original design's own phased roadmap
v2.0	TD-09 (i18n), TD-04 (offline write queue — the install/update half already shipped), TD-11 (photo classification)
Ongoing	TD-10 (expand test depth every time a new bug class is found in production)